

Computational Linear Algebra for Large Scale Problems

Homework

Politecnico di Torino
Simone Domenico Morandi, 336974
A.Y. 2025-2026

January 3, 2026

Python code for the following homework can be found at [1].

Homework 1 - PageRank

Overview

For the execution of this homework, we first define the function that allows us to compute the scores of each page. Then, we first focus on applying the simplified version of the PageRank algorithm onto two different small webs of pages and then onto a dataset of 6012 pages. Then we apply the full algorithm, with the damping factor m , onto the same webs and dataset. Lastly, we solve two exercise: the first one will consist in adding some particular connections to an existing web and see how the scores change. The second one focuses on the different results and computation times obtained by setting different values of m .

Function: *pagerank*

The main function in our code, *pagerank*, is able to apply the PageRank algorithm to both dense and sparse matrices and compute the eigenvector corresponding to the largest eigenvalue, that is $\lambda_{max} = 1$, according to the theoretical results, if the iteration matrix is column stochastic.

We define the function as follows.

Listing 1: Definition of pagerank function.

```
def pagerank(A, m, tol=1e-12, max_iter=1000):
```

It is sufficient to pass as input the adjacency matrix and the desired value of m , the damping factor; the tolerance and the maximum number of iteration, that represent our stopping criteria, are set a priori. An essential aspect of this function, is that the iteration matrix for the power method must be column stochastic in order for the algorithm to converge to the greatest eigenvalue. In order to achieve this, we must treat carefully those webs that

present dangling nodes, i.e. pages that point to no other pages. These dangling nodes are represented by columns full of zeros in the adjacency matrix A .

Our function prepares A for the power method by treating separately sparse and dense matrices.

Listing 2: Normalization procedure for the columns of A .

```
n = A.shape[0]
if sparse.issparse(A):
    A = A.tocsc()
    col_sums = np.asarray(A.sum(axis=0)).ravel()
    # Handling dangling nodes.
    dangling = (col_sums == 0)
    col_sums[col_sums == 0] = 1
    A = A.multiply(1.0 / col_sums) # Preserving the sparsity.
else:
    col_sums = A.sum(axis=0)
    dangling = (col_sums == 0) # Needed for m != 0.
    col_sums[col_sums == 0] = 1
    A = A / col_sums
```

As we can see, if A is sparse, the function converts it into CSC format¹ and normalizes the columns² while keeping track of the columns that sum to zero (with the array "dangling"). In order to avoid division for zero, the sum of the columns representing a dangling node is temporarily set to 1. On the other hand, if the matrix is dense a standard column normalization is performed, while keeping track of dangling nodes. After handling both types of matrix, the power method is applied. If $m = 0$, then the iteration to compute the eigenvector r_{new} is simply: $r_{new} = Ar$, where r is the eigenvector computed at the previous iteration, otherwise the function applies the full PageRank iteration, while accounting for dangling nodes contribution, that is:

Listing 3: Power Method iteration.

```
dangling_mass = r[dangling].sum()
r_new = (1 - m) * (A @ r) + m*s + (1-m)*dangling_mass*s
```

Note that s is a vector with entries $\frac{1}{n}$, where n is the size of the matrix. The auxiliary array *dangling_mass* is computed in order to account for dangling nodes. By doing this, the sum on the columns of the iteration matrix is always equal to one, even if we are looking at a dangling node.

The function stops the power method if the norm 1 of difference between the old and the new eigenvector is less than the tolerance. Even though this is not the cheapest stopping criterion in terms of computational cost, we avoided computing the eigenvalue since we know from theoretical results that in the end is going to be equal to 1. In the code there is a commented³ part where we compute $\|Ar - r\|_1$ in order to further confirm the convergence of

¹This format is the best choice to perform modifications on a sparse matrix.

²The method *.multiply* executes operations column-wise while maintaining the sparsity of the matrix, so it is a convenient choice for the normalization procedure.

³We commented this part out in order to not increase the computational cost of the function.

the method, this residual is always close to zero in every application of the function. Note also that all normalization of the eigenvectors required by the power method are executed relatively to the norm 1 in order to reduce the computational costs.

Now we have a function that handles sparse and dense matrices in an efficient way and computes the eigenvector containing the scores of each page of the web, while accounting for dangling nodes, which is the most likely and general case.

Remark

Later on in the report, we are going to apply only once the function with $m = 0$ to a dataset presenting dangling nodes. In this case, the matrix A will be column sub-stochastic, because the dangling nodes columns will sum to 0, and therefore $\lambda_{max} \leq 1$, so the convergence is not theoretically guaranteed. Though, exercise number four in [3] proposes to handle this particular case by looking for a normalized eigenvector in norm 1, in order to get suitable results; which is exactly what we do in the power method part inside *pagerank*. Therefore, we manage to get feasible results in any circumstance: dense and sparse matrices both with and without dangling nodes.

Simple PageRank algorithm

We can start by applying *pagerank* to simple webs and with $m = 0$. First of all, we must define the webs of pages with outgoing links that we are going to use for our first application of the algorithm. We call *Web 1* the following graph.

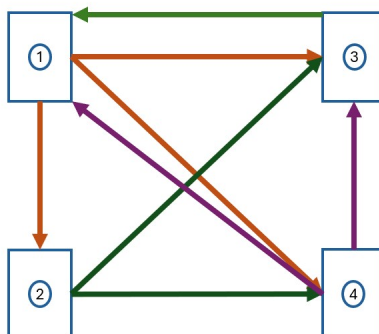


Figure 1: Graphic representation of Web 1.

And, similarly, we call *Web 2* the following graph.

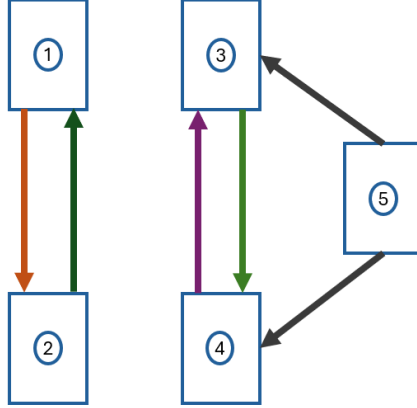


Figure 2: Graphic representation of Web 2.

An important feature of Web 2 is that it has a page that is pointed by no other pages, this will later show some limitations of this simple model. From these two graphs we can easily retrieve the adjacency matrices A_1, A_2 , respectively. Note that for this algorithm, given an adjacency matrix A , we have that:

$$\begin{cases} A_{i,j} = 1 & \text{if page } j \text{ points to page } i, \\ A_{i,j} = 0 & \text{otherwise.} \end{cases}$$

Thanks to our previously defined function *pagerank*, it is possible to compute the importance scores of Web 1 and Web 2; we get the following results:

Page	Value
1	0.387
2	0.129
3	0.290
4	0.194

Table 1: Importance scores of Web 1, $m = 0$.

Page	Value
1	0.2
2	0.2
3	0.3
4	0.3
5	0

Table 2: Importance scores of Web 2, $m = 0$.

We first notice that results for Web 1 are the same presented in [3], while the results for Web 2 are inherently wrong, since with this algorithm we find that page 5 has no importance. That is because the simplified version of the algorithm does not give importance to pages that are pointed by no other pages; this represent a limit for this model, as anticipated before. In order to compare scores computations with the full algorithm later on, we can also apply *pagerank* to the dataset *hollins.dat* (which can be found at [1]).

In order to retrieve the adjacency matrix for this dataset, we define a function, *build_A*, that, given the file name, reads through it and construct the matrix; this can be easily achieved since we know the file format. Moreover, it is convenient to store the matrix as sparse in the

LIL format, since web pages are sparsely connected. Once we have the adjacency matrix for the dataset A_{dat} , we can compute the importance scores. Since there are 6012 pages with 23875 possible connections we only displays the relevant results.

We find that 5224 pages have an importance score of zero and the most important page is the number 5456 with a score of 0.018. Again, we see that all of the pages that have no ingoing links have an importance of zero, as seen in table (2).

Full PageRank Algorithm

In this section we apply *pagerank* with $m \neq 0$ to Web 1, Web 2, comparing the results with the ones showed in [3], and the dataset. We start with Web 1 and Web 2.

Page	Value
1	0.368
2	0.142
3	0.288
4	0.202

Table 3: Importance scores of Web 1, $m = 0.15$.

Page	Value
1	0.2
2	0.2
3	0.285
4	0.285
5	0.03

Table 4: Importance scores of Web 2, $m = 0.15$.

Numerical results are identical to the ones proposed in [3] and we are now able to compare importance between subwebs, thanks to the perturbation caused by m .

Applying the full algorithm to the dataset gives us different numerical results as well. In fact, we do not have pages that have importance equal to zero, as expected; moreover, we find that page 2 is the most important with a score of 0.0199. Since the dataset is quite large, the difference between the computed scores is more evident and this can be seen in the next figure, where we plot the logarithmic scale of the scores for $m = 0$ and $m = 0.15$ with respect to the sorted pages.

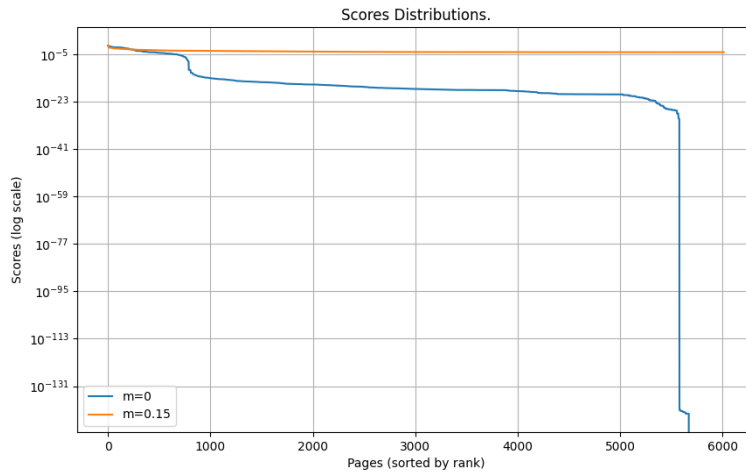


Figure 3: Comparison between scores for $m = 0$ (blue) and $m = 0.15$ (orange).

It is clear that scores distribution for $m = 0$ is not smooth and there is a disparity in terms of magnitude of the scores, while the results for $m = 0.15$ are smoother and more uniform. This is due to the fact that if there is no damping factor, the page with no outgoing links tend to 'trap' the ranking of the other pages.

It is important to remark that we showed the results for $m = 0.15$; changing the damping factor leads to different results. Lastly, if $m = 0$ we get the results obtained with the simple PageRank algorithm.

Exercise 11-12

Exercise 11

Consider Web 1, with the addition of a page 5 that links to page 3, where page 3 also links to page 5.

Calculate the new ranking by finding the eigenvector of M that has positive components summing to one. Use $m = 0.15$.

We are now asked to modify Web 1 and applying the PageRank algorithm to find the importance scores. It is important to notice that according to the notation used in [3], A is the iteration matrix for $m = 0$ and M is the one for $m \neq 0$ ⁴. It is helpful to graphically represent the new modified Web 1 (we will refer to it as *Web 1-a*).

⁴That, in the article, does not account for dangling nodes

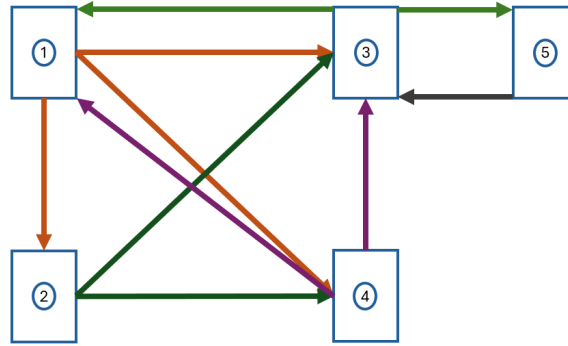


Figure 4: Graphic representation of Web 1-a.

Its adjacency matrix will have an additional row corresponding to the pages that points to page 5 and the row corresponding to page 3 will have an additional 1 in the fifth column. As done previously, we can apply *pagerank* with $m = 0.15$ and we get the following results:

Page	Score
1	0.237
2	0.097
3	0.349
4	0.138
5	0.178

Table 5: Importance scores of Web 1-a, $m = 0.15$.

If we compare these results with those in table (3), we can see that the addition of a reciprocal links between page 3 and 5, made page 3 the most important one, lowering the scores of all the other pages. Thus, with this model, it would be possible to boost the rank of one page using reciprocal linking.

Exercise 12

Add a sixth page to Web 1-a that links to every page of the web, but to which no other page links. Rank the pages using A , then using M with $m = 0.15$, and compare the results.

We are once again asked to modify our web, this one will be named *Web 1-b* and its graph is showed below.

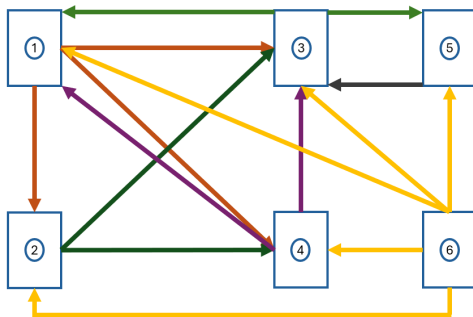


Figure 5: Graphic representation of Web 1-b.

The adjacency matrix is a 6×6 matrix with the sixth row being full of zeros. We can now compute the importance scores using *pagerank* with both $m = 0$ and $m = 0.15$.

Page	Value
1	0.245
2	0.082
3	0.367
4	0.122
5	0.184
6	0

Table 6: Importance scores of Web 1-b, $m = 0$.

Page	Value
1	0.231
2	0.095
3	0.340
4	0.135
5	0.174
6	0.025

Table 7: Importance scores of Web 1-b, $m = 0.15$.

As expected, the simple PageRank algorithm does not give importance to pages that are not pointed by any other pages. Comparing table (5) with the results obtained in table (7) with $m = 0.15$, we can see that the addition of a page pointing to all pages do not change the importance scores in a relevant way, that is because its vote is equally distributed.

Exercise 17

How should the value of m be chosen?

How does this choice affect the rankings and the computation time?

To answer these questions it is convenient to select some values of m and testing the algorithm onto a plausible dataset, for example the one given by *hollins.dat*. While executing the algorithm, we will record the times of execution and the importance scores vector as well as the variance of the final eigenvector, so that we can track how the ranking of different pages varies as m varies.

We will use the following values for the damping factor:

$$m_{\text{values}} = [0.01, 0.1, 0.3, 0.5, 0.7, 0.85, 0.9, 0.99],$$

in order to observe what changes occur throughout all the domain of m .

Time of execution

Using *time* library, we are able to measure the time of execution of the function *pagerank* and we can easily plot these times with respect to the value of m , as shown below.

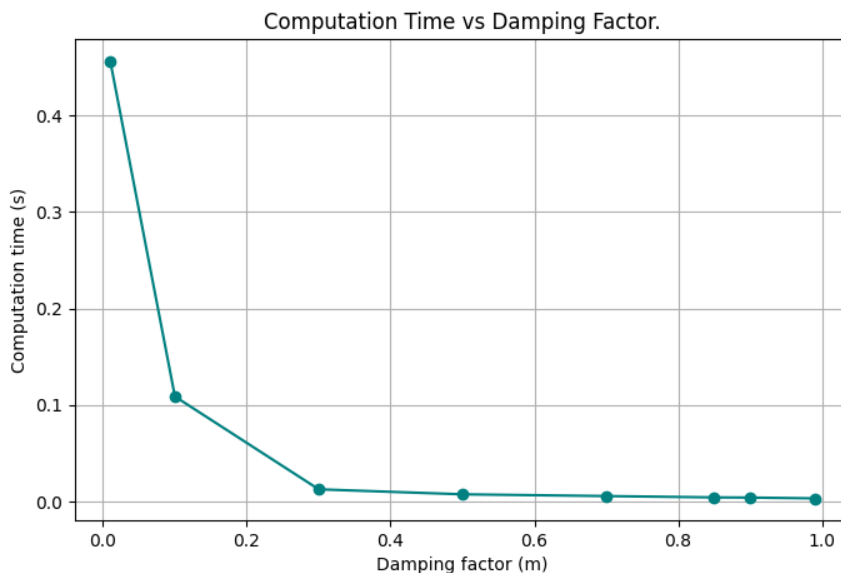


Figure 6: Time of execution with respect to m values.

As we can see, as m approaches 1, computations take less time. This is consistent with the definition of m , because if it is close to one, that means that algorithm is frequently jumping from page to page, visiting the entirety of the dataset faster. Viceversa holds for values of the damping factor that are close to zero. Note that even with m close to zero, computation times are relatively fast, that is because dataset's dimension is rather small compared to the size of an actual, realistic web.

Variance

Moreover, we can keep track of the variance of the scores vector with respect to different values of the damping factor.

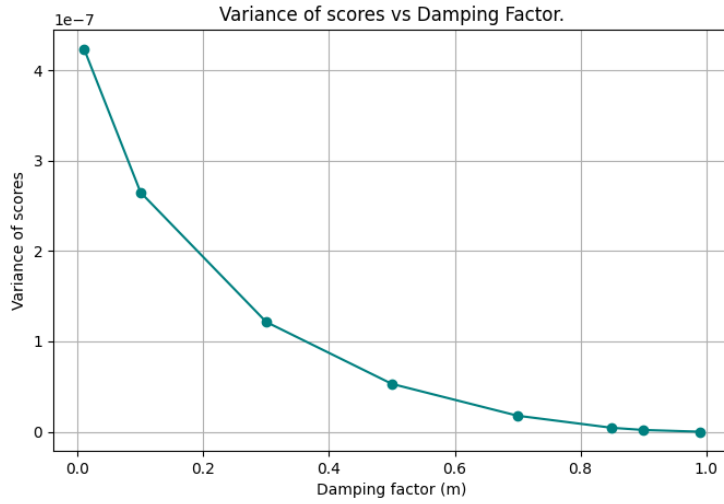


Figure 7: Variance of scores vector with respect to m values.

As expected, the variance value tends to zero as m approaches one, that is because when $m = 1$, the PageRank algorithm assign an importance of $\frac{m}{n} \simeq 1.66 \times 10^{-4}$ to each page in the dataset, so there is no variance in the scores. It is important to notice that even for small values of m we have a variance of order 10^{-7} . This is acceptable because, as said before, we are analyzing a dataset of relatively small dimension with a small dispersion.

Changes in ranking

Finally, we can choose some random pages to be tracked throughout the different evaluation of the ranking scores. Results of a single run are shown in the following image.

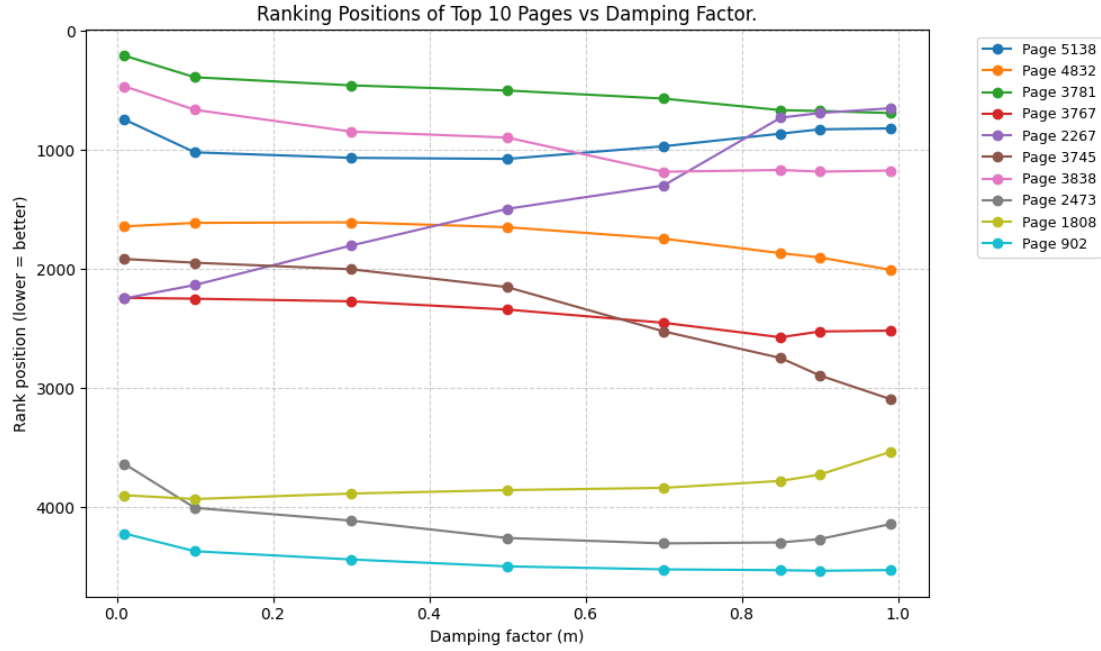


Figure 8: Tracked importance of 10 random pages as m varies.

It is clear that rank can change a lot as m changes⁵, so we can have very different results and the choice of the damping factor is therefore crucial.

⁵This can be seen by running the code multiple times and tracking different pages.

Homework 2 - Principal Component Analysis

This homework is covered into a notebook on *Jupiter*, which can be found at [1].

Homework 3 - Spectral Clustering

Overview

In the following homework we will first highlight the limitations of Kmeans method for clustering, then we will define a function that applies the spectral clustering algorithm and allow us to obtain better results. Finally, we will apply this algorithm to different dataset and we will explore the structures of some significant datasets via the graph Laplacian and its spectrum.

Limitations of Kmeans

As done in [4], we first apply the Kmeans method to three different datasets: Gaussian Mixture Model (GMM), concentric circles and anisotropic data. We can easily generate the first two datasets thanks to the functions *make_blobs* and *make_circles*. For the anisotropic data, we define a function (*make_bars*) that generates vertical bars using random uniformly distributed data. After that, we can apply the Kmeans method via the built-in function *Kmeans* to our datasets and we get the following results.

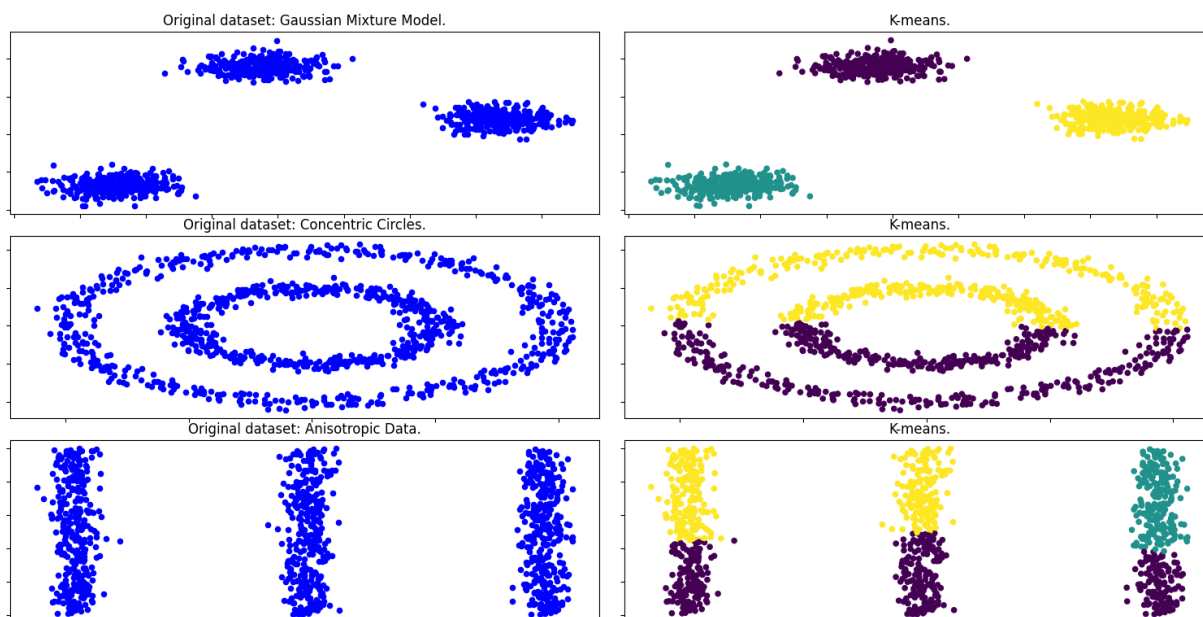


Figure 9: Plots of datasets and their Kmeans clusterizations, respectively.

As we can see, for the concentric circle and the anisotropic data, the clusterization procedure is not optimal, even though the separation between different clusters is explicit. We can improve these results by implementing a spectral clustering algorithm.

Function: `spectral_clustering`

The main function in the code is *spectral_clustering*, defined as follows:

Listing 4: Spectral clustering function.

```
def spectral_clustering(X, k, n_neighbors, sigma,
                        Norm, random_state):
```

This function will take as input the dataset we want to divide into clusters (X) and the number of clusters we want obtain (k). The number of neighbors, if it's different from *None*, is passed along with σ to create a σ -neighborhood graph; otherwise a fully connected graph is created. Both weights matrices are symmetrized in the end to get feasible results. After that, the function computes the graph Laplacian, defined as $L = D - W$, where D is a diagonal matrix containing the sum over the rows of W , the weight matrix defined earlier, or the normalized graph Laplacian (we can call them both L) depending on the value of the boolean *Norm*. Now the function enters in the core of the spectral clustering algorithm: eigenvalues and the k smallest eigenvectors of L are computed via *eigh* method⁶; the latter ones are stored column-wise into a matrix U . Lastly, the Kmeans method is applied and fit to the matrix U . The seed used in the Kmeans method is set by *random_state*. The function returns the labels obtained from the Kmeans method, the eigenvalues of the graph Laplacian and the matrix U .

Spectral clustering

With the previously defined function we can try different types of clusterization procedures depending on the type of graph we create at the beginning and what type of graph Laplacian we want to use. We can now show some results applied to the previous datasets.

⁶This method is the most efficient one for symmetric matrices since we need all the spectrum of L and not just few eigenpairs.

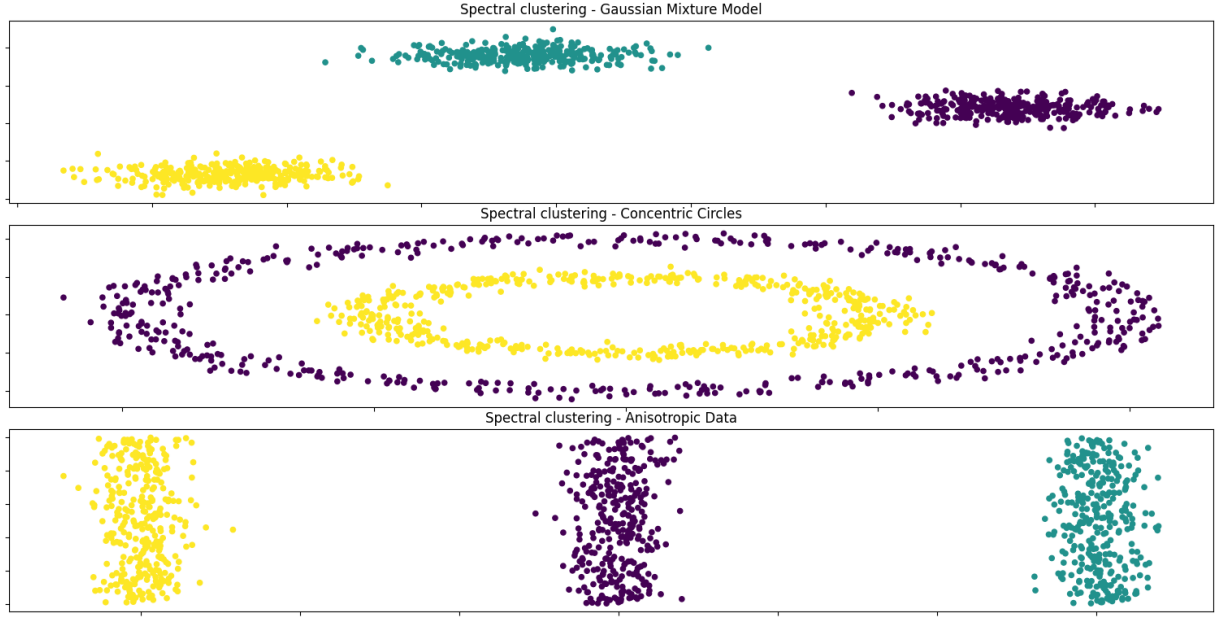


Figure 10: Spectral Clustering:
Number of neighbors = [10, 10, 15], sigma = [1.0, 0.2, 1.5], Norm = False.

With the parameter set in figure (10), we obtain a perfect clusterization for each datasets. If we maintain the number of neighbors and sigma values fixed, while using the normalized graph Laplacian $\left(L = D^{-\frac{1}{2}}WD^{-\frac{1}{2}}\right)$, we obtain a perfect clusterization as well⁷. We now try to use a fully connected graph for the procedure, thus the number of neighbors is always set to *None* for all datasets.

⁷The choice of using the normalized Laplacian graph is preferable when there is a high disparity in terms of magnitude among data.

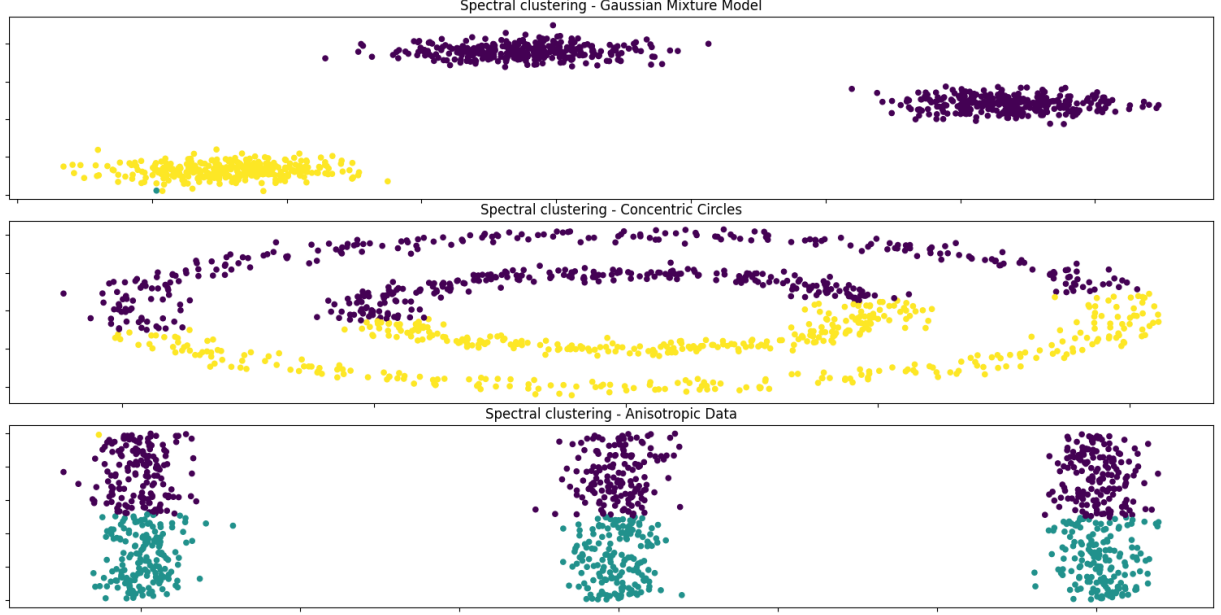


Figure 11: Spectral Clustering:
Number of neighbors = *None*, $\sigma = [1.0, 0.2, 1.5]$, Norm = False.

As we can see, the choice of a fully connected graph is not optimal for both concentric circles and anisotropic data. Results are similar with the normalized Laplacian graph. This tells us that parameters for the clusterization procedure must be chosen accordingly in order to obtain satisfactory results.

As one could expect, the normalization procedure affects the spectrum of L , as mentioned in [4]; we can take a look at the eigenvalues for (normalized) graph Laplacian with the parameter set in figure (10) and (11), respectively.

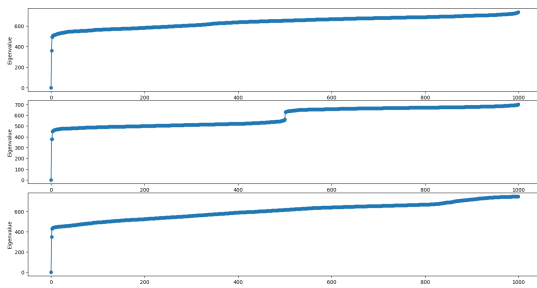


Figure 12: Eigenvalues of L for GMM, Concentric Circles and anisotropic data, respectively, with σ -neighborhood graph.

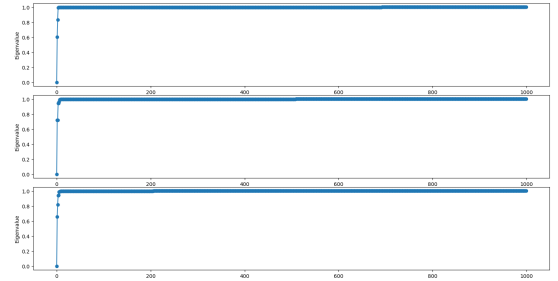


Figure 13: Eigenvalues of normalized L for GMM, Concentric Circles and anisotropic data, respectively, with fully connected graph.

As we can see, the eigenvalues for the normalized graph Laplacian belong to range $[0, 1]$, as one could expect, since L is normalized, while the eigenvalues of the not normalized L can assume very different values.

Exercise 1

Verify that discrete sine and cosine functions are the eigenvectors of graph Laplacian associated to n -cycle.

If we consider a n -cycle, we can build its adjacency matrix, that will present the following structure:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 & 0 & 1 \\ 1 & 0 & 1 & \cdots & 0 & 0 & 0 \\ 0 & 1 & 0 & \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 0 & 1 & 0 \\ 0 & 0 & 0 & \cdots & 1 & 0 & 1 \\ 1 & 0 & 0 & \cdots & 0 & 1 & 0 \end{bmatrix}$$

We can easily compute the Laplacian graph; its eigenvalues will be:

$$\lambda_k = 2 - 2 \cos \frac{2\pi k}{n}, k \in [0, n-1],$$

and the corresponding eigenvectors of L are:

$$\phi_k = \left[\cos\left(\frac{2\pi k}{n}\right), \cos\left(\frac{4\pi k}{n}\right), \dots, \cos\left(\frac{2(n-1)\pi k}{n}\right) \right]^T.$$

Now we want to verify that the discrete sine and cosine functions are the eigenvectors of this graph Laplacian.

In order to do so, we can define the discrete sine and cosine functions as follows.

Listing 5: Discrete sine and cosine functions.

```
j = np.arange(n)
xi_k = np.cos(2*np.pi*k*j/n)
psi_k = np.sin(2*np.pi*k*j/n)
```

Then, we can compute the following quantities to see whether these functions are eigenvectors of L :

$$\|L\xi - \lambda_k\xi\|, \quad \|L\psi - \lambda_k\psi\|,$$

for every value of k . We do get that the both norms go to zero, thus proving our statement. We can also see that the functions and the eigenvalues match graphically.

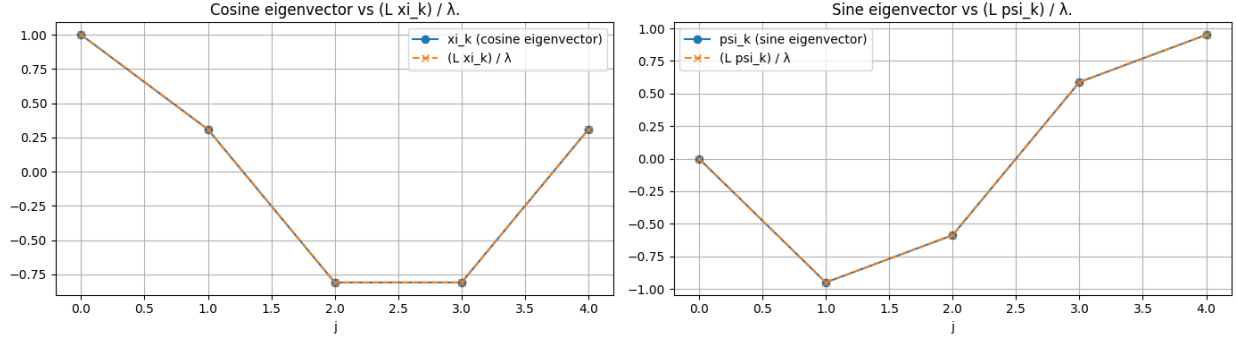


Figure 14: Comparison between eigenvalues and discrete cosine and sine function for $k = 4$.

If we take different values of k , functions continue to match perfectly.

Exercise 2

Find the spectrum of the graph Laplacian associated to the n -hypercube.

For this exercise we used some theoretical results about the hypercube illustrated in [2] (section I and II). Starting from $n = 2$, i.e. a square, we can easily build the next dimensional hypercube by taking two squares and connecting each node to the corresponding one in the other square⁸, that gives us a cube as shown below.

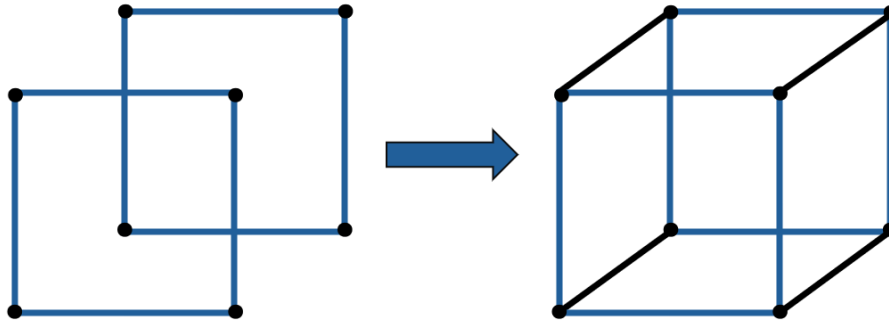


Figure 15: Passage from 2-hypercube (square) to 3-hypercube (cube).

If we recursively repeat the process illustrated in figure (15), we can get the generic n -dimensional hypercube. In terms of adjacency matrices, we get the following recursive scheme:

⁸That would be the cartesian product between two graphs, as explained in [2], Section I, chapter E.

$$Q_n = \begin{bmatrix} Q_{n-1} & I \\ I & Q_{n-1} \end{bmatrix},$$

where I is the identity matrix and Q_{n-1} is the adjacency matrix corresponding to the $(n-1)$ -hypercube; of course $Q(0) = [0]$. Note that $I \in \mathbb{R}^{(n-2) \times (n-2)}$ since each hypercube has 2^n nodes. Consequently, we are able to compute the general graph Laplacian corresponding to Q_n as follows:

$$L(Q_n) = \begin{bmatrix} L(Q_{n-1}) + I & -I \\ -I & L(Q_{n-1}) + I \end{bmatrix}.$$

Next step is to define a recursive function that numerically computes $L(Q_n)$.

Listing 6: Function for graph Laplacian of the n -hypercube.

```
def laplacian_hypercube(n):
    # Base block.
    if n == 1:
        return np.array([[1, -1],
                        [-1, 1]], dtype=float)
    L_prev = laplacian_hypercube(n-1) # Recursive call.
    # Assembling final Laplacian.
    I = np.eye(2**(n-1))
    L = np.block([
        [L_prev + I, -I],
        [-I, L_prev + I]
    ])
    return L
```

By calling *laplacian_hypercube* with dimension n , the function will recursively call itself with parameter $n-1$ until $n=1$. Then, it starts building the block that represents the weight matrix in a 1-dimensional cube (a segment with two vertices), finally it recursively builds the final block matrix, representing the n -hypercube

We know from [2] that the eigenvalues of Q_n are:

$$\sigma(Q_n) = \{-n, -n+2, -n+4, \dots, 0, \dots, n-4, n-2, n\},$$

with multiplicity:

$$\binom{n}{0}, \binom{n}{1}, \dots, \binom{n}{n}.$$

Consequently, the eigenvalues of $L(Q_n)$ are:

$$\sigma(L(Q_n)) = \{0, n+2, \dots, 2n\},$$

with the same multiplicity as the eigenvalues of Q_n . In order to confirm these results, we can compute Q_5 by calling *laplacian_hypercube* and we get:

$$\sigma(L(Q_5)) = \{0, 2, 3, 6, 8, 10\},$$

as expected from theoretical results. Moreover, by plotting each eigenvalues with its multiplicity, we can clearly see that the theoretical results are confirmed.

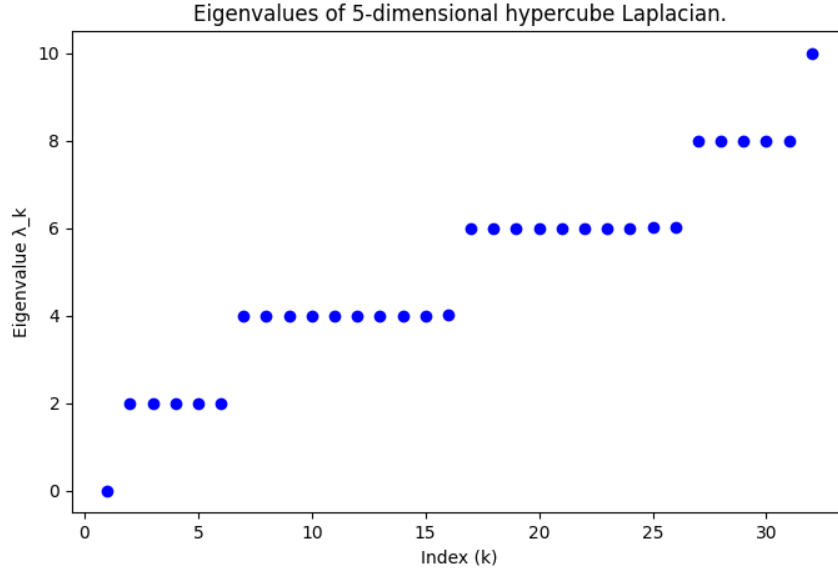


Figure 16: Eigenvectors of $L(Q_5)$ with their multiplicity.

Remark

In the code of this homework, available at [1], there are some other exercises that have not been reported here because they were a mere confirmation of the theoretical results presented in [4]. Though, they were needed to get a full understanding of the topic.

References

- [1] *Homework python code*, 2025. Github repository.
- [2] S. F. F. III, *Spectral Graph Theory Of The Hypercube*, B.S., United States Military Academy, 1999.
- [3] T. L. KURT BRYAN, *The 25,000,000,000 eigenvector, the linear algebra behind google*.
- [4] S. LING, *Spectral clustering, graph laplacian*, Lecture Notes, (2020).